REPOSITÓRIO · PWA-HUB-FASE1 · LEVANTAMENTO 100%

Mapeamento completo do ecossistema PWA Hub Concretrack

Inventário técnico de todas as aplicações, backends, scripts de automação e documentação encontrados no repositório e nas duas VPS de produção, produzido para servir de base ao redesenho e reconstrução da plataforma como um sistema único e coerente.

Gerado em 28 ago 2026 **Repositório** pwa-hub-fase1 **Branch analisada** fix/dashboard-scroll-tablet-500

Domínio usina.concretrack.com.br

31 APPS & MÓDULOS	**2** VPS DE PRODUÇÃO	**3×** ERP+DASHBOARD DUPLICADO POR UNIDADE	**6** APPS SEM AUTENTICAÇÃO
1 APP COM TESTES AUTOMATIZADOS DE UI (E2E)	**17** APPS REGISTRADOS EM APP_PERMISSIONS		

RESUMO EXECUTIVO

O repositório `pwa-hub-fase1` não é uma aplicação única — é um **hub** de ~20 aplicações e scripts, construídos em momentos e com padrões diferentes ao longo do tempo, todos servindo a operação de uma usina de concreto (Concrefer/Concretrack). Vão de PWAs estáticas de página única com backend em Google Apps Script até um sistema full-stack NestJS + React com testes end-to-end, passando por agentes Python de automação que geram relatórios e disparam WhatsApp, e um projeto embrionário de LIMS de laboratório ainda não construído.

A aplicação mais madura e ativa é o **Mapa de Concretagem** (app.js com 12.326 linhas), seguida por **Concrejet**, um sistema à parte (apesar do nome) para produção de injeção plástica, com arquitetura NestJS/React exemplar e a única suíte de testes e2e do repositório. A maior parte das demais PWAs segue um padrão mais simples: HTML/JS estático + Google Apps Script como "backend", sem build, sem testes, com autenticação inconsistente.

Fora do repositório versionado, duas VPS de produção hospedam mais oito aplicações não rastreadas em nenhum git: o PCP Concrefer e o GED Concrefer na VPS 31.97.241.84, e — na VPS 2.25.163.32 — **três cópias quase idênticas** de um "ConcreTrack ERP" (FastAPI) + "Dashboard MES" (Flask), uma por unidade fabril (Noroeste, Pederneiras, Ribeirão Preto). O achado mais relevante para o seu objetivo de unificação: **já existe uma quarta aplicação, ConcreTrack Unificado**, publicada em produção desde julho/2026, que começou exatamente esse trabalho de consolidação multi-unidade — ver seção dedicada e as recomendações finais.

ARQUITETURA & ILHAS DE BACKEND

Quatro sistemas de backend distintos coexistem hoje, cada um com seu próprio banco de dados e forma de acesso — todos, confirmou-se, rodando na mesma VPS (31.97.241.84). Qualquer reconstrução precisa decidir se consolida tudo em um só.

① Supabase (fbvvdyirhtgvycullsqy)

Postgres + Auth + Edge Functions (Deno). Hub de login/RBAC (`users_app`, `app_permissions`), produção (`producao`, `vw_formas_status`), dashboards analíticos v1, tabela nova `prog_s3_s4`. Consumido direto do browser (anon key) por Mapa de Concretagem, Controle Estatístico, auth/, e via REST pelo Agente PCP e scripts de defeitos.

② ConcreTrack ERP

FastAPI + PostgreSQL própria (`tbl_massadas`), rodando em VPS via systemd + Nginx. Serve `/concretrack/api/relatorios/*` (fechamento mensal, consumo). Documentado só por engenharia reversa (RELATORIO_FECHAMENTO...md) — não está neste repositório.

③ dashboard-usina (MES)

Flask/Gunicorn + PostgreSQL própria (`tbl_massadas`, `tbl_pcpsetor`), também em VPS. Serve o card "STATUS OPERACIONAL" ao vivo em dautomacao.com/usina. Também documentado por engenharia reversa.

④ PCP Concrefer

FastAPI + PostgreSQL própria (`pcp_concrefer`) em `/opt/pcp-concrefer`, mas também lê Supabase (mesma `producao`) e Google Sheets — a única ilha que já fala com as outras três ao mesmo tempo. Ver mapeamento completo em PCP Concrefer.

Além disso, cinco apps legados (Cartas Traço, Checklist, Dosagem, QR Concreto, Slump/Flow) usam cada um seu próprio Google Apps Script Web App ligado a uma planilha Google Sheets específica — nenhum schema ou API compartilhada entre eles. PedeRH e Controle Estatístico de Concreto têm cada um seu próprio Postgres + Express, independentes de tudo isso. Concrejet tem seu próprio Postgres + NestJS, também isolado.

> **Ponto de atenção —** o arquivo `.env.example` da raiz do hub descreve variáveis de um serviço (`resultado_ensaios_db`, porta 8010) que não corresponde a nenhum código presente na raiz — provavelmente resíduo de outro projeto (`PROMPT_CODEX_GITHUB_VPS_POSTGRESQL.md` documenta o provisionamento desse serviço à parte, "resultado-ensaios-usina").

NÚCLEO DO HUB

PWA Hub — Shell raiz

PRODUÇÃO

/ (index.html, login.html, splash.html, status.html, alertas.html…)

HTML/CSS/JS vanilla | sem build | Supabase | Service Worker

Página estática que apresenta a grade de ícones (`.hub-grid`) linkando para cada sub-aplicação, seja como caminho relativo (`./qr-concreto/index.html`) seja como iframe externo (Dashboard Usina em `dautomacao.com/usina/embed`). Sem framework, sem bundler — scripts carregados via `<script defer>`.

FLUXO	splash.html (start_url do manifest) → index.html (grade) → login.html → sub-apps. status.html é um changelog/status standalone; alertas.html lê histórico de notificações de um IndexedDB (ct-alerts-db) alimentado pelo service worker.
BACKEND	Supabase (auth + dados) + Edge Functions (auth-login, change-password, admin-users) + microsserviço de push separado em dautomacao.com/etiquetas (VAPID).
SERVICE WORKER	Cache versionado (atualmente ct-hub-v41-mapa-bypass), estratégia network-first para HTML/navegação e cache-first para estáticos; bypassa explicitamente chamadas a script.google.com e as pastas mapa-concretagem* (que têm SW próprio); implementa Web Push e log de alertas em IndexedDB.
MANIFEST.JSON	Nome "Concre HUB", atalhos para QR Concreto, Setor Count, Slump/Flow.
PACKAGE.JSON	Nome pwa-hub, deps: @supabase/supabase-js, xlsx; scripts amarram testes de dosagem-concreto, seed-master.mjs e o bump de versão do Mapa de Concretagem.

> **Achado —** ícones em `assets/img/` para "controle de cimento" e "paradas" existem mas não aparecem como blocos na grade atual — possíveis funcionalidades planejadas/descontinuadas a validar no redesenho.

auth/ — módulo de autenticação e RBAC compartilhado

PRODUÇÃO

/auth (8 arquivos)

`Supabase Auth` `Postgres RLS` `Edge Functions`

Biblioteca central de sessão/RBAC carregada por quase todas as sub-aplicações (`config.js`, `client.js`, `guard.js`, e em algumas `hub.js`). Senhas e sessões são geridas **exclusivamente** pelo Supabase Auth — a tabela `users_app` nunca guarda senha.

ARQUIVOS	config.js (URL/anon key + flag authEnabled), client.js (wrapper + RPC my_access_context), guard.js (gate de página: redireciona sem sessão, expira por sessionMaxAgeSeconds=28800, força troca de senha no primeiro login, checa data-app-key contra permissões), hub.js (renderiza tiles conforme permissão), login.js, change-password.js, admin-users.js (CRUD de usuários, restrito a access_level==='master'), ui.css.
SEGURANÇA	Front usa somente a chave pública anon; operações administrativas passam por Edge Functions com SUPABASE_SERVICE_ROLE_KEY (nunca exposta no cliente). Hospedagem estática no GitHub Pages permanece pública — a proteção real vem do guard de rota + RLS no Postgres, não de esconder arquivos.
ESTADO ATUAL	authEnabled: false — o gate está hoje **desligado** em todo o hub, embora o fluxo completo esteja implementado e pronto para reativar.
COBERTURA	Nem todas as apps carregam o guard: Slump/Flow, Cartas Traço e setor_count não o referenciam.

Supabase — backend central do hub

PRODUÇÃO

/supabase (migrations/, functions/) · projeto fbvvdyirhtgvycullsqy

Postgres | Deno Edge Functions | RLS

Banco único compartilhado por Mapa de Concretagem, auth/, Agente PCP, scripts de defeitos e (parcialmente) o front-end do Concrejet para sessão. Mesma SUPABASE_URL usada em todos esses pontos.

MIGRAÇÃO	CONTEÚDO
202606290001_auth_hub.sql	Schema de auth: users_app, login_logs, app_permissions (seed com 17 apps), login_attempts (rate limit), funções is_master() / my_access_context().
202608200001_add_vibrado_to_producao.sql	Campo vibrado em producao (usado pela lógica de qualidade do Agente PCP).
202608200002 / 003 …dashboard*	Schema de analytics dos dashboards v1: funções de setor, dashboard_retrabalho_eventos_v1 com custo e aprovação.
202608200004 …contract_fix_v1.sql	Função recursiva para extrair rótulos "não" de payloads JSONB de checklist/defeito.
202608280001_create_prog_s3_s4.sql	Nova tabela prog_s3_s4 (programação diária Setores 3/4) — a mais recente, corresponde ao commit "Cria tabela de programação do Setor 3".

EDGE FUNCTIONS: auth-login (rate limit por hash de IP+usuário, verify_jwt=false), admin-users, change-password (ambas exigem sessão + requireMaster()), _shared/security.ts (CORS, clientes admin/user, política de senha 12+ caracteres).

APPS REGISTRADOS: dashboard-usina, qr-concreto, slump, parada-usina, massadas, setor-botoes, dosagem-concreto, controle-cimento, controle-agregados, checklist-usina, mapa-concretagem, etiquetas-cp, controle-estatistico, manual-dosagem, alertas, cartas-traco-unidades, pcp, admin-usuarios — lista oficial de 17 apps geridos pelo RBAC (nem todos têm código correspondente encontrado no repo, ex.: parada-usina, controle-cimento, controle-agregados, etiquetas-cp).

APLICAÇÃO PRINCIPAL

Mapa de Concretagem + versão -teste

PRODUÇÃO · FLAGSHIP

/mapa-concretagem · /mapa-concretagem-teste

`PWA vanilla JS` `app.js: 12.326 linhas` `Supabase realtime` `Express opcional (fotos)`

A aplicação mais complexa e ativamente desenvolvida do repositório. Acompanha o ciclo de vida completo de fôrmas/moldes pré-moldados: **programação PCP** (amarelo) → **liberação de formas** (azul) → **concretagem/produção** (verde), com relatórios cruzando os três estágios, captura de fotos de inspeção, dashboards de defeitos, controle de manutenção de mandril circular e a nova tabela de programação do "Setor 3".

FRONTEND	PWA estática, app.js monolítico (12.326 linhas) + index.html (1.696 linhas). Bundla localmente supabase.js, Chart.js e plugin de datalabels (sem CDN). SW próprio e versionado independentemente do hub (ex.: v4.61), explicitamente excluído do cache do SW raiz.
BACKEND	Supabase direto do browser para todo o CRUD/realtime; API externa pcp.concretrack.com.br/api/programacao (com fallback usina.concretrack.com.br/api/programacao para o Setor 3); API Node/Express própria e opcional (server.js, pacote concretrack-storage-api) para upload de fotos via SFTP (usa multer, sharp, ssh2-sftp-client) — esse mesmo server.js também pode servir o hub inteiro como estático.
DADOS EXTRA	supabase-formas-manutencao.sql (schema de manutenção de moldes), data/produto-volumes.json (315 registros peso/volume, gerado 28/08/2026 a partir de cadastro Excel).
VERSÃO -TESTE	Cópia de homologação explícita (rotulada "[TESTE]" na grade do hub) — app.js, index.html, manifest.json, sw.js todos divergentes da produção.
DOCUMENTAÇÃO PRÓPRIA	HANDOFF_TECNICO.md, documentacao_completa.html, guia-funcionalidades-v1-v123.html — fontes primárias valiosas para o redesenho.

> **Achado —** um único arquivo `app.js` de 12k linhas concentra praticamente toda a lógica de negócio da aplicação mais crítica do hub — forte candidato a modularização em qualquer reconstrução.

SISTEMAS FULL-STACK

Concrejet — "ConcreTrack Injeção"

PRODUÇÃO · MAIS MADURO

/concrejet (backend/, frontend/, docs/, imports/)

`NestJS 10 + TypeScript` `React 18 + Vite` `PostgreSQL 16 + TypeORM` `Playwright e2e`

Apesar do nome do repositório sugerir concreto, este sistema substitui uma planilha Excel de ~160 abas usada para controlar **produção por injeção plástica**: cadastros, apontamentos, ocorrências, resinas/lotes, blendas, estoque e indicadores (OEE). Desenvolvido em fases documentadas (Fase 0 a 7+), com trilha completa de ADRs e handoff.

BACKEND	NestJS + TypeORM com **migrações SQL manuais** (não synchronize, não Prisma) para suportar EXCLUDE USING gist/tstzrange (evita apontamentos sobrepostos por máquina). Auth dupla: cookie JWT httpOnly + Argon2id para admin; matrícula+PIN → bearer token para operador de chão de fábrica.
MÓDULOS	auth, organizacao, usuarios, producao-base, auditoria, cadastros, producao, ocorrencias, estoque, analytics, importacao, health.
FRONTEND	React/Vite PWA para tablet (paisagem, chão de fábrica) + painel desktop admin. Zustand + React Query + react-hook-form/zod + axios; offline-first com Dexie (IndexedDB) e fila de sincronização (outbox).
TESTES E2E	Playwright: activation-login, admin-auth, dashboard-scroll-tablet, fases-5-7-admin, resin-lots — **única suíte e2e de todo o repositório**.
DOCUMENTAÇÃO	ADRs (0001 stack, 0002 ORM/migrações, 0003 auth), arquitetura.md, modelo-dados.md, regras-negocio.md, calculos-oee.md, mockups HTML aprovados em docs/referencia-ui/, e um handoff.md vivo (log de sessão por agente).
DEPLOY	docker-compose.yml (Postgres + API); frontend via Vite dev server exposto na LAN para tablets; sem pipeline de build/deploy de produção documentado além de vite build.

TRABALHO EM CURSO (BRANCH ATUAL)

Rate limiting tornado configurável via env; correção de bug onde `current-by-device` reexibia apontamento já concluído; correção de vazamento de sessão entre admin e operador (uma chamada global `/auth/me` derrubava a sessão do operador); specs e2e atualizados para IP de LAN dinâmico.

PedeRH

PROTÓTIPO AVANÇADO

/PedeRH (backend/, frontend/, scripts/)

`Node.js + Express 4` `PostgreSQL (SQL puro)` `HTML/JS vanilla`

Sistema interno de solicitações e fluxos de RH ("Peça RH"): advertências, atrasos, banco de horas, horas extras, dispensas, férias, movimentação de pessoal, recrutamento — mais um módulo completo de entrega/rastreio de EPI com assinatura digital.

BACKEND	server.js → app.js, camadas controllers/services/routes/middlewares, schema.sql + seed.sql (sem ferramenta de migração), fallback em JSON (src/data/db.json via utils/store.js).
BANCO	Tabelas: usuarios, solicitacoes, solicitacoes_timeline, solicitacoes_anexos, epi_itens, epi_estoque, epi_entregas, epi_entregas_itens; usa JSONB para campos polimórficos por tipo de solicitação.
API	/api/auth, /api/solicitacoes (+ PDF), /api/dashboard, /api/users, /api/epi (itens/estoque/entregas/assinatura/PDF), /api/credenciais.
AUTH	**Frágil**: login valida bcrypt mas retorna token = user.id em texto puro — sem JWT, sem expiração, sem revogação. Guards de perfil (requirePerfis, requireSuperAdmin) existem, mas o "token" é literalmente o ID do usuário.
TESTES	Nenhum.

> **Achado —** `package.json` descreve o projeto como "Backend do protótipo PWA RH"; histórico git mostra um único commit "Initial PedeRH production snapshot" — importado como snapshot pronto, não desenvolvido iterativamente neste repositório.

Controle_est_concre

PRODUÇÃO (BACKEND INTERNO)

/Controle_est_concre (migrations/, src/, tests/)

`Node.js + Express 5` `PostgreSQL (SQL puro)` `PM2 + Nginx`

API de sincronização entre ConcreTrack e o app QR Concreto: persiste resultados de rompimento de corpos de prova de concreto, sincroniza a partir do Google Sheets e calcula estatísticas diárias/mensais de resistência contra faixas-alvo.

ESTRUTURA	src/api (v1 legado + v2), src/core (config, database, cache, security — sem auth), src/services (inclui sync do Google Sheets), src/repositories, src/jobs/backfill.js, src/workers/analytics_worker.js (fila de recálculo).
BANCO	rompimentos (base), + migração 001_analytics_schema.sql: faixas_resistencia, rompimentos_eventos (auditoria), rompimentos_atuais, rompimentos_grupos (pares CP1/CP2), estatistica_diaria/mensal, fila_recalculo.
DEPLOY	Browser → Nginx (:8086) → Express (:3086, interno) → Postgres (:5432 local); ecosystem.config.cjs (PM2), provision_remote.sh para provisionar VPS.
AUTH	Nenhuma no nível de aplicação — apenas allow-list de CORS e rate limit por IP em memória; segurança depende de rede (Nginx + firewall).
TESTES	tests/analytics.test.js, tests/controle_estatistico.test.js via node --test — testes de integração reais (sobem o app Express, mockam DB/fetch).

DocFlow

SCAFFOLD — NÃO CONSTRUÍDO

/docflow (frontend/)

`React + Vite (existente)` `FastAPI + Celery (planejado)`

README descreve uma plataforma SaaS de Gestão Eletrônica de Documentos (GED) para ambientes industriais regulados — multi-tenant, com Supabase Auth/RLS, backend FastAPI+JWT+Celery, deploy Ubuntu+Nginx+Gunicorn+Systemd. **Nada disso existe no repositório** além de um esqueleto de frontend.

O QUE EXISTE	Apenas frontend/: App.jsx renderiza um Layout com uma única página Dashboard.jsx estática (um card fixo "Docs Ativos: 1.284"), sem chamadas de API, sem roteamento, sem uso real de zustand/axios apesar de estarem no package.json.
O QUE FALTA	Não existe backend/, docker-compose.yml, workflow de deploy ou scripts — todos referenciados no README mas ausentes.
INTEGRAÇÃO	Não está linkado na grade do hub, no manifest nem no cache do service worker raiz — totalmente desconectado do resto do hub hoje.

> **Achado —** README termina com "Este README é gerado automaticamente pela Antigravity" — reforça que é boilerplate aspiracional, não um produto em construção ativa.

GED Concrefer

PRODUÇÃO · MAPEADO VIA SSH NA VPS

VPS 31.97.241.84 · /opt/ged_concrefer · ged.concretrack.com.br

`Node.js 20 + Express` `Next.js 14 + Tailwind` `PostgreSQL 16` `JWT` `Google Drive API` `Docker Compose`

Não está listado em nenhum lugar do repositório `pwa-hub-fase1` — foi localizado diretamente na VPS que hospeda o PCP Concrefer e outros serviços "concretrack-*". É um sistema de **Gestão Eletrônica de Documentos (GED)** real, completo e ativo (containers Docker rodando há 4 semanas): controle documental com versionamento, aprovação eletrônica, armazenamento no Google Drive e controle de acesso por perfil — cobrindo o mesmo domínio conceitual do DocFlow (que é só scaffold), mas totalmente implementado e em produção fora deste repositório.

> **Achado —** a VPS também guarda um documento antigo (`/root/GED-CONCREFER_DOCUMENTACAO_COMPLETA.md`, maio/2026) que descreve "GED-CONCRETRACK" como um módulo dentro do PedeRH (Node/Express, porta 3010, banco `pwa_rh`) — uma nomeação/implementação anterior e diferente da atual. O Nginx e os containers Docker de hoje confirmam que `ged.concretrack.com.br` aponta agora para esta aplicação própria e independente (portas 3000/4000, banco `ged_concrefer`), não mais para o módulo de RH. Nomeação ambígua a esclarecer na reconstrução.

BACKEND	backend/src: controllers (documentController, approvalController, authController, userController, readLogController, calibrationController, rncController — não conformidades, taskController), services (driveService, documentService, pdfService, emailService, rbacService, auditService, alertService, storageService, passwordPolicy), middlewares (auth JWT, roles), db/migrations.sql + seed.sql. Deps: express, pg, jsonwebtoken, bcryptjs, googleapis, multer, pdf-lib, qrcode, nodemailer.
FRONTEND	Next.js 14 (App Router) + Tailwind, contexts/AuthContext, hooks/useAuth + useDocuments, client Axios em services/.
ARMAZENAMENTO	Arquivos vão para o **Google Drive** via Service Account, em 4 pastas fixas que espelham o ciclo de vida do documento: 00_VIGENTES, 01_EM_REVISAO, 02_OBSOLETOS, 03_MODELOS (replicadas localmente em backend/uploads/{vigentes,em_revisao,obsoletos,modelos}, mais pastas extras calibrations e tasks não documentadas no README).
PERFIS DE ACESSO	ADMIN (tudo), QUALIDADE e ENGENHARIA (upload/aprovação/leitura), LABORATORIO (upload/leitura), LEITOR (somente leitura + confirmação de ciência) — modelo de RBAC voltado a controle de qualidade (ISO/SGQ), coerente com o módulo de Dashboards v1 do hub.
FLUXO DE VERSIONAMENTO	Upload → EM_REVISÃO → aprovação → VIGENTE; nova versão do mesmo documento move a anterior para OBSOLETO e reinicia em EM_REVISÃO com revisão incrementada (Rev 00 → Rev 01); usuários recebem e-mail e devem confirmar ciência (log de leitura registrado).

MÓDULOS EXTRA	**RNC** (registro de não conformidade) e **Calibração** — controllers/rotas próprias, não mencionados no README principal. **Tarefas** (/tasks): tabelas tasks, task_comments, task_history, task_participants, task_attachments, task_document_links.
API	POST /api/auth/login, GET /api/auth/me, GET/POST /api/documents, POST /api/documents/:id/upload (nova versão), GET /api/documents/:id/versions, GET /api/approvals/pending, POST /api/approvals/:versionId/approve|reject, POST /api/read-logs, GET /api/users (admin).
DEPLOY	Docker Compose: 3 containers — ged_postgres (16-alpine, migrations/seed via docker-entrypoint-initdb.d), ged_backend (porta 4000, volume de uploads + credencial Google montada read-only), ged_frontend (porta 3000, build com NEXT_PUBLIC_API_URL). Nginx faz proxy: / → 3000 (frontend, com suporte a upgrade/websocket), /api e /uploads/ → 4000 (backend), com client_max_body_size 50M para upload de documentos.
CREDENCIAIS SEED	admin@concrefer.com.br / Admin@2024 — README recomenda troca imediata; mesmo padrão de risco (senha fixa documentada) observado no PCP Concrefer.

PWAS UTILITÁRIAS (PADRÃO LEGADO: HTML + APPS SCRIPT)

Cinco aplicações de página única, cada uma com seu próprio backend em Google Apps Script ligado a uma planilha Google Sheets específica — sem schema ou API compartilhada entre si.

QR Concreto

PRODUÇÃO

/qr-concreto (27 arquivos)

`HTML/JS + leitor QR` `Google Apps Script`

Leitura de QR Code em corpos de prova cilíndricos (CPs) para rastreio de ensaios de rompimento — atalho padrão/primeiro do manifest PWA. É a app mais "polida" como PWA instalável: conjunto completo de ícones e splash screens iOS (48×48 até 1024×1024).

BACKEND	Google Apps Script Web App próprio.
AUTH	Usa o guard /auth/ do hub.

> **Achado —** arquivos de ícone duplicados (`icon-192x192.png` e `icon-192x192.png.png`) sugerem artefato de exportação a limpar.

Slump/Flow

LEGADO, SEM AUTH DO HUB

/slump (7 arquivos)

HTML/JS + fetch clima | Google Apps Script | GitHub Pages/Jekyll

Registro de ensaios de abatimento (slump)/espalhamento, com busca automática de clima e cruzamento de riscos por data/hora. Único app do hub com workflow próprio de deploy Jekyll/GitHub Pages — indício de que nasceu como projeto standalone antes de ser absorvido pelo hub.

BACKEND	Google Apps Script com autenticação por token compartilhado (TOKEN em ambos os lados).
AUTH DO HUB	Não carrega o guard /auth/.

Dosagem de Concreto

PRODUÇÃO · ÚNICO COM TESTES UNITÁRIOS

/dosagem-concreto (15 arquivos + testes)

HTML/JS modular | node --test | 3× Google Apps Script

Calculadora de traço/dosagem: cálculo por argamassa, dosagem científica e dosagem por argamassa, mais um manual completo (`manual.html`). O maior arquivo HTML estático do repositório (`index.html`: 12.430 linhas).

TESTES	tests/argamassa-calculo.test.js, dosagem-cientifica.test.js, dosagem-por-argamassa.test.js — lógica de cálculo desacoplada da UI e coberta por asserções numéricas reais; único app do repo com testes automatizados de lógica.
BACKEND	Três Apps Script separados: custo MCC, dosagem, fornecedores/granulometria.

Checklist da Usina

PRODUÇÃO, COM RESÍDUO DE VERSÕES ANTIGAS

/checklist-usina (+ apps-script/)

HTML/JS | Mini-PWA própria | Google Apps Script

Checklist operacional digital (limpeza, silos, linhas de aditivo, sensores, temperatura da água — 30+ itens). `index.html` é um shim de redirecionamento para `checklist_industrial_usina_mobile_v3.html`, indicando pelo menos duas versões anteriores superadas.

BACKEND	Google Apps Script gravando numa planilha específica (doPost).
AUTH	Usa o guard /auth/ (data-app-key="checklist-usina").

> **Achado —** arquivos de Apps Script duplicados em dois locais (`apps-script/Code.gs` e `AppsScript_checklistUsina.gs`) — limpeza incompleta após migração v1→v3.

Cartas Traço

LEGADO, ÓRFÃO

/carta-traco (2 arquivos)

HTML + Chart.js | Google Apps Script

Visualizador de registros/referências de traço de concreto para o laboratório. Está no cache do service worker do hub, mas **não aparece como bloco** na grade de `index.html` — link órfão/legado. Não carrega o guard de autenticação.

Controle Estatístico

PRODUÇÃO

/controle-estatistico (7 arquivos)

`HTML/JS` `REST API real` `Mini-PWA própria`

Exibe resultados/relatórios de controle estatístico de rompimentos de concreto. Diferente dos demais legados: consome uma **API REST real** (`concretrack.com.br/api/controle-estatistico/v2/rompimentos-grupos`), com adaptação de formato v1→v2 e cache em `localStorage`.

AUTH `Usa o guard /auth/ do hub.`

Setor Botões & Setor Count

PROTÓTIPOS UTILITÁRIOS

/setor_botoes (1 arquivo) · /setor_count (2 arquivos)

`HTML/JS de página única`

Duas ferramentas pequenas e relacionadas de contagem de produção por setor no chão de fábrica. `setor_botoes`: UI de botões grandes coloridos para lançar contagem manualmente, sem arquivo Apps Script próprio (provavelmente fala direto com Supabase). `setor_count`: contagem via leitor QR, tema escuro — **não carrega** o guard `/auth/` e só é acessível via atalho do manifest, não pela grade principal do hub.

PCP — atalho/redirecionador

SHIM, NÃO É APP

/pcp (3 arquivos)

`HTML redirect`

Não é uma aplicação real — é um "atalho PWA para o login do PCP" (texto literal do manifest): mostra uma tela de carregamento e redireciona para `https://pcp.concretrack.com.br/pages/login.html`. O verdadeiro sistema de Planejamento e Controle de Produção vive fora deste repositório, em outro domínio — mapeado a seguir via acesso direto à VPS.

PCP Concrefer — aplicação real por trás do atalho

VPS 31.97.241.84 · /opt/pcp-concrefer · pcp.concretrack.com.br

PRODUÇÃO · MAPEADO VIA SSH NA VPS

`Python 3.11 + FastAPI` `SQLAlchemy + PostgreSQL` `JWT + bcrypt` `Bootstrap 5.3 + JS puro` `Nginx + Systemd`

Sistema completo de **Planejamento e Controle da Produção** para a fábrica de postes de concreto, hospedado na mesma VPS que os demais serviços "concretrack-*". É uma aplicação própria e mais tradicional (FastAPI monolítico + Postgres dedicado) do que o padrão Supabase-direto-do-browser usado no restante do hub, mas também fala com Supabase e Google Sheets como fontes de dado secundárias.

BACKEND	app/main.py + routers em app/api/routers/: auth, usuarios, setores, pcp, dashboard, produtos, alocacao_formas, teste_programacao. Camadas core (config/database/security), models (SQLAlchemy: producao, produto, setor, usuario, alocacao_formas, mapeamento_forma(+acessorio/+poste), teste_programacao), schemas (Pydantic), services (auth_service, dashboard_service, alocacao_formas_service, sheets_service) e repositories/google_sheets_repository.py.
DADOS	**Três fontes coexistindo**: banco Postgres dedicado (pcp_concrefer, via DATABASE_URL), Supabase (SUPABASE_URL/SUPABASE_ANON_KEY/SUPABASE_PRODUCAO_TABLE no .env — provável leitura da tabela producao do mesmo projeto central usado pelo Mapa de Concretagem) e Google Sheets (credenciais de service account + SPREADSHEET_ID, via gspread).
AUTH	JWT (python-jose) + bcrypt (passlib), 4 níveis de acesso — ADMIN, PCP, PRODUCAO, CONSULTA — com matriz de permissão explícita por ação (criar programação, lançar produção, lançar mão de obra, gerenciar usuários). README documenta usuários padrão com senhas fixas (admin@concrefer.com/Admin@123 etc.) — risco de segurança se não trocados em produção.
FRONTEND	HTML + Bootstrap 5.3 + JS puro + Chart.js, ~19 páginas em frontend/pages/: login, primeiro-acesso, dashboard, programacao(+alex,+final), producao, maodeobra(+rateio), comparacao, etiquetas, formas-por-setor, acompanhamento-mensal(+setor-34), apontamento-mega, pcp-diario, teste-programacao, usuarios. Nomes como programacao-alex/programacao-final/teste-programacao sugerem iterações/experimentos paralelos ainda não limpos.
API	POST /api/login, GET /api/dashboard, GET /api/volume-total, CRUD de setores, formas, programacao, producao, maodeobra, usuarios (admin). Swagger habilitado em /api/docs (ENABLE_DOCS=true).
MÓDULO INTEGRADO — ARMADURAS	Sub-aplicação independente **PCP Armaduras (Ribeirão Preto)** em /opt/pcp-armaduras, também FastAPI, rodando como serviço systemd separado (pcp-

	armaduras.service, porta 8022) e exposta via Nginx sob /armaduras/ (proxy de API + uploads). Estrutura própria de app/api, app/models, app/core e frontend próprio em frontend/src/.
INFRAESTRUTURA	Nginx com TLS (Let's Encrypt), HSTS e headers de segurança (X-Frame-Options, X-Content-Type-Options, Permissions-Policy); dois processos Uvicorn systemd com Restart=always: pcp-concrefer.service (porta 500, 2 workers, rodando como www-data) e pcp-armaduras.service (porta 8022).

> **Achado —** existe uma **terceira instância** praticamente idêntica em `/var/www/PCPCONCREFER`, com seu próprio serviço systemd ativo (`pcpconcrefer-api.service`, porta 8020) — mas nenhum bloco do Nginx encontrado faz proxy para a porta 8020. É um deployment mais antigo (abril/2026, contra agosto/2026 do `/opt/pcp-concrefer`) que continua rodando em segundo plano sem estar exposto publicamente — candidato a desligar/remover na reconstrução.

> **Achado —** a raiz `/root/pwa-hub` desta mesma VPS contém um checkout completo deste próprio repositório `pwa-hub-fase1` — ou seja, a VPS 31.97.241.84 é também onde o hub PWA e o agente PCP×Produção (`pcp_producao_agent`) rodam em produção, ao lado do PCP Concrefer, ConcreTrack ERP e dashboard-usina. Todos os "sistemas externos" documentados por engenharia reversa na seção anterior vivem nesta mesma máquina.

AUTOMAÇÃO & RELATÓRIOS (AGENTES PYTHON)

Agente PCP × Produção

PRODUÇÃO · AGENDADO

/pcp_producao_agent

`Python` `Supabase REST` `Google Sheets (gspread)` `Webhook WhatsApp`

Job em lote diário que compara o plano oficial de produção (PCP, em Google Sheets) contra a produção real (tabela `producao` no Supabase), classifica cada item (Realizado/Parcial/Excedente/Não Produzido/Não Programado), gera um relatório HTML executivo e envia um resumo por WhatsApp.

ARQUIVO	PAPEL
`agent_pcp_producao.py`	CLI/orquestrador (`--data`, `--dry-run`, `--check-config`)
`config.py`	Carrega `.env`; expõe URLs/keys Supabase, Google, WhatsApp
`concretrack_client.py`	Cliente REST PostgREST (produção, `vw_formas_status`); catálogo de códigos de poste
`google_sheets_client.py`	Lê aba "5 - PCP DIÁRIO - PADRÃO" via `gspread` + service account
`comparator.py`	Motor de regras de comparação + análise de qualidade (vibrado/exsudado/segregado)
`html_report.py`	Gera relatório HTML responsivo (35KB, gráficos SVG feitos à mão)
`whatsapp_sender.py`	Envia resumo via webhook estilo Evolution API

AGENDAMENTO — Documentado (README) via crontab (07:00, `--data ontem`) ou Agendador de Tarefas do Windows — não há artefato de agendamento real dentro do repo para *este* agente especificamente (diferente do agente de defeitos, que tem um `.bat`).

SAÍDA — `relatorios/pcp-producao/relatorio_pcp_producao_YYYY-MM-DD.html` (7 relatórios presentes, jul–ago 2026).

> **Achado —** existe uma cópia paralela quase idêntica deste agente em `relatorios/scripts/pcp_producao_agent/`, com seu próprio `.env`/logs — aparenta ser um snapshot arquivado, não uma segunda instância viva.

Agente de Defeitos de Concreto

PRODUÇÃO · ÚNICO COM AGENDAMENTO CONFIRMADO

/scripts (raiz)

`Python` `Supabase` `Task Scheduler (.bat)`

Agente diário (D-1) que gera um relatório de "Defeitos de Concreto": classifica registros de `producao` por tipo/status, calcula percentuais e volumes estimados de defeito por setor.

ARQUIVOS	`daily_concrete_defects_agent.py` (orquestrador diário, reaproveita `generate_defeitos_concreto_report.py` para o resumo mensal), `seed-master.mjs` (bootstrap do primeiro usuário master), `AppsScript_unificado.gs` (automação dentro da própria planilha de PCP).
AGENDAMENTO REAL	`run_daily_concrete_defects_agent.bat` — **única evidência concreta de agendamento automatizado** no repositório (chama o script com `--data ontem`, grava log).
SAÍDA	`relatorios/defeitos-concreto/diario/`.

relatorios/ — saída consolidada

MISTURA DE ATIVO + ARQUIVADO

/relatorios (pcp-producao/, defeitos-concreto/, scripts/)

Diretório de saída dos dois agentes acima, mais `relatorios/scripts/`, uma mini-área à parte com `relatorio_defeitos_concreto_mtd_v4.py` e dois subdiretórios que parecem **snapshots congelados**: uma cópia antiga do agente PCP e um script standalone de 138KB (`analise_comparativa_defeitos_LT-PRO_2026.py`) comparando "App vs. Planilha" para Setores 1–2.

SISTEMAS EXTERNOS (FORA DO REPOSITÓRIO, DOCUMENTADOS POR ENGENHARIA REVERSA)

ConcreTrack ERP & Dashboard Usina (MES)

FORA DO REPOSITÓRIO

VPS externo · não versionado neste repo

FastAPI | Flask/Gunicorn | PostgreSQL própria

Dois serviços web vivos em produção, cada um com seu próprio Postgres (tabela `tbl_massadas` compartilhada conceitualmente, mas bancos distintos do Supabase). Documentados neste repo apenas como especificação para reimplementação:

CONCRETRACK ERP	FastAPI (/opt ou /var/www/concretrack-erp), roteador relatorios.py, serve /concretrack/relatorios/fechamento e /concretrack/relatorios/consumo — fechamento mensal, comparação MoM/YoY, classificação de concreto por prefixo de fórmula. Documentado em RELATORIO_FECHAMENTO_CONCRETRACK_IMPLEMENTACAO.md.
DASHBOARD-USINA (MES)	Flask/Gunicorn (/var/www/dashboard-usina, serviço systemd), endpoint GET /api/usina/dashboard, lê tbl_massadas + tbl_pcpsetor, calcula taxa (m³/h), gap vs. meta das 17h, status ATRASADO/ADIANTADO/NO PRAZO. Documentado em docs/levantamento-mes-dashboard-usina.md.
INFRA DE REFERÊNCIA	PROMPT_CODEX_GITHUB_VPS_POSTGRESQL.md documenta o padrão de provisionamento usado nesses serviços: VPS + Postgres isolado por app + usuário Linux não-root + systemd + Nginx reverse proxy — runbook para um projeto adicional, resultado-ensaios-usina.

UNIDADES REGIONAIS — VPS 2.25.163.32

Uma **segunda VPS** (2.25.163.32), totalmente separada da VPS do PCP Concrefer/GED (31.97.241.84), hospeda três instâncias praticamente idênticas do mesmo par de aplicações — uma para cada unidade fabril: **Noroeste** (`noro.concretrack.com.br`), **Pederneiras** (`pederneiras.concrefer.com.br`) e **Ribeirão Preto** (`ribeirao.concretrack.com.br`). Mapeado 100% via SSH direto na VPS.

O padrão triplicado: ConcreTrack ERP + Dashboard MES por unidade

/var/www/{supervisorio_noro, supervisorio_ribe, concretrack-pederneiras} + /var/www/dashboard-{noroeste,pederneiras,ribeirao}

CÓDIGO TRIPLICADO — MAIOR ALVO DE CONSOLIDAÇÃO

Cada unidade roda **dois serviços Python independentes**, sempre com o mesmo par de tecnologias, mesmo entrypoint (`uvicorn api.main:app` / `gunicorn app.app:app`) e — na prática — o mesmo código-fonte copiado e adaptado unidade a unidade (confirmado por `diff` entre as pastas de Noroeste e Ribeirão: divergências pontuais em `main.py`, `auth.py`, `relatorios.py`, `tracos.py`, o resto é idêntico):

CAMADA	NOME INTERNO (README)	STACK	ESCOPO
ConcreTrack ERP (API + frontend "supervisório")	`concretrack-api`	Python + FastAPI + Uvicorn, PostgreSQL própria por unidade (`supervisorio_db`), frontend HTML/JS servido como estático	ERP modular completo: cadastros, cartas-traço, agregados, cimento, estoque, custos, PCP, produtos, relatórios, autenticação própria, e **CEC — Controle Estatístico de Concreto** (leitura de QR de corpo de prova, mesmo domínio do Controle_est_concre da outra VPS, aqui reimplementado do zero) + **leitura de nota fiscal por IA** (OpenAI/Gemini/Groq/HuggingFace/Ollama configurável)
Dashboard de Produção (MES)	`Dashboard Concrefer v2`	Python + Flask + Gunicorn, SQLAlchemy, mesma `supervisorio_db` Postgres, ingestão automatizada de Excel	Monitoramento em tempo real (auto-refresh 2 min): ciclos de produção (OK/Aviso/Crítico), tempo de batida, paradas técnicas, qualidade (exsudação), produtividade m³/h, comparativo manhã/tarde. Convive com um **dashboard legado** (`legacy_dashboard.py`, ~2.800 linhas) que lê Excel diretamente, mantido em paralelo à versão moderna orientada a banco.

Os dois serviços de cada unidade compartilham o mesmo banco Postgres local (tabelas como `tbl_massadas` e `tbl_leiturasetor` confirmadas via histórico de comandos deixado no servidor) — exatamente o mesmo modelo de dados documentado para o "dashboard-usina" da VPS principal (`docs/levantamento-mes-dashboard-usina.md`), porém aqui replicado três vezes com bancos totalmente isolados entre unidades.

UNIDADE	DOMÍNIO	PASTA ERP	PORTA API	PASTA DASHBOARD	PORTA DASHBOARD
Noroeste	`noro.concretrack.com.br`	`supervisorio_noro`	8010	`dashboard-noroeste`	8002

Pederneiras	pederneiras.concrefer.com.br	concretrack-pederneiras	8011	dashboard-pederneiras	8001
Ribeirão Preto	ribeirao.concretrack.com.br	supervisorio_ribe	8012	dashboard-ribeirao	8003

Achado — as pastas de cada ERP contêm dezenas de subpastas `backups/codex-AAAAMMDD-HHMMSS-*` (ex.: `codex-20260803-075112`, `codex-20260821-132651-estoque-frete`, `codex-auth-filter-20260827-153536`) — evidência de que cada unidade vem sendo evoluída de forma independente por agentes de IA (Codex) diretamente em produção, unidade por unidade, o que explica e acelera a divergência de código entre elas.

Achado — há também arquivos corrompidos/residuais dentro de `supervisorio_noro/` (nomes de arquivo literalmente iguais a trechos soltos de CSS e comandos `psql`, ex.: `:18px 18px 0 0 !important`, `ol id rgba(...))` — resíduo de algum comando com redirecionamento mal-formado executado direto no servidor. Não afeta a aplicação em si, mas é sujeira a limpar antes de qualquer migração.

Unidade Noroeste

PRODUÇÃO

noro.concretrack.com.br

ERP	/var/www/supervisorio_noro — serviço supervisorio_noro.service (porta 8010). Rotas: agregados, auth, custos, estoque, health, massadas, pcp, produtos, qrcp, relatorios, tracos. IA de nota fiscal configurada via GEMINI_API_KEY. Inclui pasta solta erp-concre/ (mini front à parte) e config de Nginx própria versionada no repo (nginx/concretrack-erp.conf).
DASHBOARD	/var/www/dashboard-noroeste — serviço dashboard-noroeste.service (porta 8002, Gunicorn 2 workers). Mesma dualidade legado/moderno das demais unidades.
FRONTEND	public/ com cadastros, cec, estoque, relatorios, trocar-senha, usuarios, mais páginas de diagnóstico (mapa-aplicacao.html, relatorio-sistema.html, test-auth.html) deixadas em produção.

Unidade Pederneiras

PRODUÇÃO

pederneiras.concrefer.com.br / pederneiras.concretrack.com.br

ERP	/var/www/concretrack-pederneiras — serviço concretrack-pederneiras.service (porta 8011). Mesma estrutura de pastas do Noroeste (api/, apps/dashboard, frontend/, public/, static/, erp-concre/). É esta pasta que serve como **raiz do domínio** (nginx root /var/www/concretrack-pederneiras/public), diferente de Noroeste/Ribeirão onde a raiz é a pasta "supervisorio".
DASHBOARD	/var/www/dashboard-pederneiras — serviço dashboard-pederneiras.service (porta 8001). Única unidade com .git/ local (repositório git próprio, não presente nas outras duas) — indício de que esta foi a unidade "modelo" a partir da qual as outras foram copiadas.
ROTEAMENTO	Nginx roteia / → ERP (raiz), /dashboard e /usina e /api/usina/ → Dashboard MES (8001), /api/ → ERP (8011), /static/ → assets do Dashboard. Rota /concretrack faz redirect permanente para / (limpeza de uma URL legada com prefixo).

Unidade Ribeirão Preto

PRODUÇÃO

ribeirao.concretrack.com.br

ERP	/var/www/supervisorio_ribe — serviço supervisorio-ribeirao.service (porta 8012). Mesma base de Noroeste, com auth.py, relatorios.py e tracos.py divergentes (evoluídos separadamente).
DASHBOARD	/var/www/dashboard-ribeirao — serviço dashboard-ribeirao.service (porta 8003).
ROTEAMENTO	Única unidade com prefixo /concretrack/ explícito e duplicado como raiz também (nginx serve o mesmo conteúdo em / e em /concretrack/, ambos apontando para supervisorio_ribe/public) — outra pequena divergência de configuração acumulada entre as três unidades.

★ ConcreTrack Unificado — a consolidação já começou

/var/www/concretrack-unificado · concrefer.concretrack.com.br · v0.3.0

EM PRODUÇÃO, ESCOPO PARCIAL — BASE IDEAL PARA CONTINUAR

FastAPI (mesma stack) | config-driven multi-unidade | 3 bancos via env var | já no ar

Achado mais importante desta VPS: já existe uma quarta aplicação, `concretrack-unificado`, criada especificamente para consolidar as três unidades em **um único serviço multi-tenant** — respondendo diretamente ao objetivo de "deixar tudo rodando em uma aplicação só". Está rodando em produção desde 24/07/2026 (`concretrack-unificado.service`, porta 8015) e já é servida no domínio "canônico" `concrefer.concretrack.com.br` — mas com escopo bem mais restrito que os três ERPs completos, ou seja, é um começo, não uma migração terminada.

ARQUITETURA MULTI-TENANT	config/units.yaml declara as 3 unidades como configuração de dados, não código: cada uma tem display_name, legacy_hosts (domínios antigos), database_url_env (nome da variável de ambiente com a connection string daquela unidade), dashboard_internal_url/api_internal_url (para onde ainda proxya o que não foi migrado) e uma lista de modules habilitados por unidade.
3 BANCOS, 1 CÓDIGO	.env define PEDERNEIRAS_DATABASE_URL, RIBEIRAO_DATABASE_URL, NOROESTE_DATABASE_URL + um AUTH_DATABASE_URL separado (schema próprio, sql/auth_schema.sql, api/core/auth_db.py) — ou seja, a unificação já resolveu a camada de **autenticação cross-unidade** antes mesmo de migrar os módulos de negócio: existe um conceito explícito de full_access_users e confirmed_multi_unit_users (usuários com acesso a mais de uma unidade) no units.yaml.
ROUTERS MIGRADOS ATÉ AGORA	api/main.py (FastAPI, título "ConcreTrack Unificado", v0.3.0) registra apenas: health, auth, config, units, qrcp, tracos, diagnostics. Ou seja, dos ~10 módulos de negócio do ERP completo (cadastros, agregados, custos, estoque, massadas, pcp, produtos, relatorios, CEC/leitura de CP), **só QR-CP e cartas-traço foram migrados** — o restante ainda depende dos serviços antigos por unidade.
FRONTEND	public/ enxuto: index.html, app.js, módulo cartas-traco/ — nada comparável ainda ao frontend completo (cadastros/estoque/CEC/relatórios) dos três ERPs individuais.
FONTE DE VERDADE DECLARADA	units.yaml define cec_reference_unit: pederneiras — ou seja, a própria equipe já decidiu que a implementação de CEC (Controle Estatístico de Concreto) da unidade Pederneiras deve ser a referência ao migrar esse módulo para o Unificado.

POR QUE ISSO MUDA O PLANO DE RECONSTRUÇÃO

> Você não está começando do zero: `concretrack-unificado` já define o padrão arquitetural certo (FastAPI único + PostgreSQL por unidade selecionado em runtime + config declarativa em YAML + auth cross-unidade) e já está publicado no domínio canônico. O trabalho que falta é essencialmente **continuar essa migração** — portar módulo por módulo (cadastros, estoque, custos, PCP, produtos, relatórios e o restante do CEC) dos três ERPs antigos para dentro do Unificado, usando exatamente o mesmo padrão de `units.yaml` já validado, em vez de desenhar uma arquitetura nova. Isso também deveria virar o destino natural do PCP Concrefer e do GED Concrefer da outra VPS, se a intenção for realmente "tudo em uma aplicação só".

DOCUMENTAÇÃO & PLANEJAMENTO

Projeto "Dashboards v1"

RELEASE CANDIDATE — NÃO IMPLANTADO

/docs (17 documentos) + mockups/

`Postgres views/RPCs` `ISO/SGQ (IQ/OQ/PQ)` `Golden dataset`

Iniciativa de refatoração de backend para os dashboards de **Produtividade, Montagem e Defeitos** dentro do Mapa de Concretagem (-teste): move o cálculo de KPIs, hoje feito ad-hoc no JS do front, para views/RPCs Postgres versionadas e testadas — necessário para validação ISO/SGQ. Estruturado em 15 fases, com auditoria de baseline, migrações aplicadas apenas no projeto Supabase de teste, RPCs novas (`rpc_dashboard_produtividade_resumo_v1`, `...montagem...`, `...defeitos...`, `...retrabalho...`, etc.), dataset-ouro para testes de contrato e um mockup visual (`dashboard-defeitos-v4.html`) ainda não reconciliado com a matriz de setores aprovada.

STATUS	**Release candidate, não em produção.** 12 pendências de homologação abertas (PH-001…PH-012): sem responsáveis/códigos SGQ definidos, sem validação visual em navegador real, sem medição de performance p50/p95, papéis de aprovação de custo indefinidos.
INDICADORES	Setores S1–S4, agregados TOTAL e S1_S2 (soma antes de calcular %, nunca média de percentuais), categoria UNCLASSIFIED para dados de qualidade duvidosa. Indicadores-chave: Taxa NC, índice de reprovação, atingimento de montagem, Pareto de defeitos.

Concretrack Lab (planejado)

APENAS PLANEJADO — NADA CONSTRUÍDO

docs/plano_implantacao_concretrack_lab.md

Plano de implantação para um **LIMS** (Laboratory Information Management System) especializado em concreto/química de laboratório — produto separado do que existe hoje, com integração planejada ao Concretrack em uma fase futura.

FASE 1 (MVP)	Perfis (Técnico, Supervisor/Engenheiro, Cliente), dashboard operacional, entrada de amostra com código único, catálogo normativo de ensaios, controle de rompimento por idade do corpo de prova (7/14/28 dias) com cálculo automático de MPa, laudos em PDF.
FASE 2	Módulo de dosagem/curva A-C (ABCP/ACI), biblioteca de normas, rastreabilidade de lote de cimento, alertas de calibração de equipamento, dashboard avançado (fck, desvio padrão, taxa de rejeição).
FASE 3	QR Code para moldes de corpo de prova, integração com o sistema de automação da usina (Concretrack), portal do cliente com envio automático de PDF por e-mail.
STACK SUGERIDA	React/Vue/Next.js + Supabase (Postgres+Auth) ou backend Node. Modelo inicial: Usuarios, Obras, OrdemServico, Amostras, Ensaios, Rompimentos.

RECOMENDAÇÕES PARA A RECONSTRUÇÃO

Síntese das observações transversais levantadas em todo o inventário, para orientar as decisões de arquitetura da nova aplicação.

1. **Adote o ConcreTrack Unificado como o alicerce da consolidação, não uma reescrita do zero.** Ele já resolve os três problemas mais difíceis de um multi-tenant: seleção de banco por unidade via `units.yaml`/env var, autenticação cross-unidade com usuários de acesso múltiplo, e um domínio canônico já publicado (`concrefer.concretrack.com.br`). O trabalho restante é portar, unidade por unidade, os módulos que faltam (cadastros, agregados, custos, estoque, massadas, PCP, produtos, relatórios e o restante do CEC) dos três ERPs antigos — usando `cec_reference_unit: pederneiras` como já decidido pela própria equipe.
2. **Depois disso, avalie trazer PCP Concrefer e GED Concrefer para dentro do mesmo Unificado.** Ambos já são FastAPI + PostgreSQL (PCP Concrefer) ou Node/Express (GED Concrefer) — tecnicamente compatíveis com o padrão de módulo do Unificado. Isso reduziria as duas VPS de produção e ~8 serviços systemd/Docker distintos a uma única aplicação com um único banco por unidade/módulo.
3. **Elimine a triplicação de código nas 3 unidades antes/durante a migração.** O `diff` entre Noroeste e Ribeirão mostrou que módulos como `auth.py`, `relatorios.py` e `tracos.py` já divergiram por causa de correções feitas isoladamente por agentes de IA em cada unidade (pastas `backups/codex-*` datadas). Cada dia sem consolidar aumenta o custo de reconciliar essas divergências — priorize migrar os módulos mais divergentes primeiro.
4. **Consolidar as três ilhas de backend do lado do hub PWA.** Supabase, ConcreTrack ERP (FastAPI) e dashboard-usina (Flask) hoje mantêm bancos `tbl_massadas`/`producao` paralelos e desincronizados. Os documentos de "levantamento"/"fechamento" no repo existem justamente para extrair a lógica desses dois últimos e sugerem que a consolidação em cima do Supabase é o caminho já sinalizado pela equipe.
5. **Padronizar autenticação.** O guard `/auth/` está pronto e é bem arquitetado (Supabase Auth + RLS + Edge Functions), mas está desligado (`authEnabled: false`) e nem todo app o carrega (Slump, Cartas Traço, Setor Count). PedeRH tem autenticação própria e frágil (token = ID do usuário). Adotar um único mecanismo de sessão/RBAC para todas as apps da nova plataforma.
6. **Substituir os backends em Google Apps Script.** Cinco apps (Cartas Traço, Checklist, Dosagem, QR Concreto, Slump) dependem de planilhas Google Sheets individuais sem schema compartilhado — frágil, sem transações, sem tipo de dado forte. Migrar para tabelas Postgres com o mesmo modelo usado pelo Supabase central.
7. **Modularizar o Mapa de Concretagem.** É a aplicação mais crítica e ativa, mas concentra 12k+ linhas em um único `app.js`. Usar Concrejet (NestJS + React/Vite, módulos bem definidos, testes e2e) como referência de padrão arquitetural para o rebuild.
8. **Elevar PedeRH e Controle_est_concre ao mesmo padrão.** Ambos têm modelagem de dados razoável mas SQL puro sem migração versionada e (no caso do Controle_est_concre) nenhuma autenticação de API. Adotar TypeORM/Prisma com migrações versionadas e autenticação de serviço a serviço.
9. **Decidir o destino do DocFlow e do Concretrack Lab.** Ambos são visão/planejamento sem implementação real — mas o GED Concrefer, achado fora deste repositório, já cobre exatamente o mesmo domínio (gestão eletrônica de documentos) e está em produção. Provavelmente faz mais sentido adotar/evoluir o GED Concrefer como o módulo de GED oficial e descartar o scaffold do DocFlow, em vez de reconstruir do zero.

10. **Trazer o GED Concrefer para dentro do repositório versionado.** Hoje ele só existe na VPS (`/opt/ged_concrefer`), fora de qualquer controle de versão acessível a este repositório — mesmo padrão de risco do ConcreTrack ERP e do dashboard-usina. Também usa uma nomeação ambígua herdada de uma implementação anterior (documentação antiga confundia "GED" com o módulo de RH do PedeRH) — vale renomear/clarificar antes da migração.
11. **Limpar deployments órfãos na VPS.** A VPS 31.97.241.84 roda hoje um quarto serviço systemd, `pcpconcrefer-api.service` (`/var/www/PCPCONCREFER`, porta 8020), que não está mais referenciado por nenhum bloco do Nginx — deployment de abril/2026 superado pelo de `/opt/pcp-concrefer` (agosto/2026), mas nunca desligado. Auditar a VPS por outros serviços "fantasma" antes de migrar.
12. **Formalizar o agendamento de automações.** Só o agente de defeitos tem artefato de agendamento real (`.bat`) no repositório; o Agente PCP depende de configuração manual fora do repo. Migrar ambos para um scheduler centralizado (cron/systemd timer) versionado junto ao código.
13. **Aproveitar o trabalho já formalizado em "Dashboards v1".** É o único módulo do hub com processo de qualidade completo (IQ/OQ/PQ, dataset-ouro, dicionário de indicadores) — usar sua metodologia de contrato de dados como modelo para os demais domínios (produção, estoque, RH) na reconstrução.

Mapeamento gerado por leitura integral do repositório `pwa-hub-fase1` (branch `fix/dashboard-scroll-tablet-500`) em 28 de agosto de 2026. Cobre 100% das pastas de aplicação identificadas na raiz e subpastas de primeiro nível. Documento de referência técnica — não modifica nenhum arquivo do repositório.